Cloud Board — 클라우드 네이티브 게시판

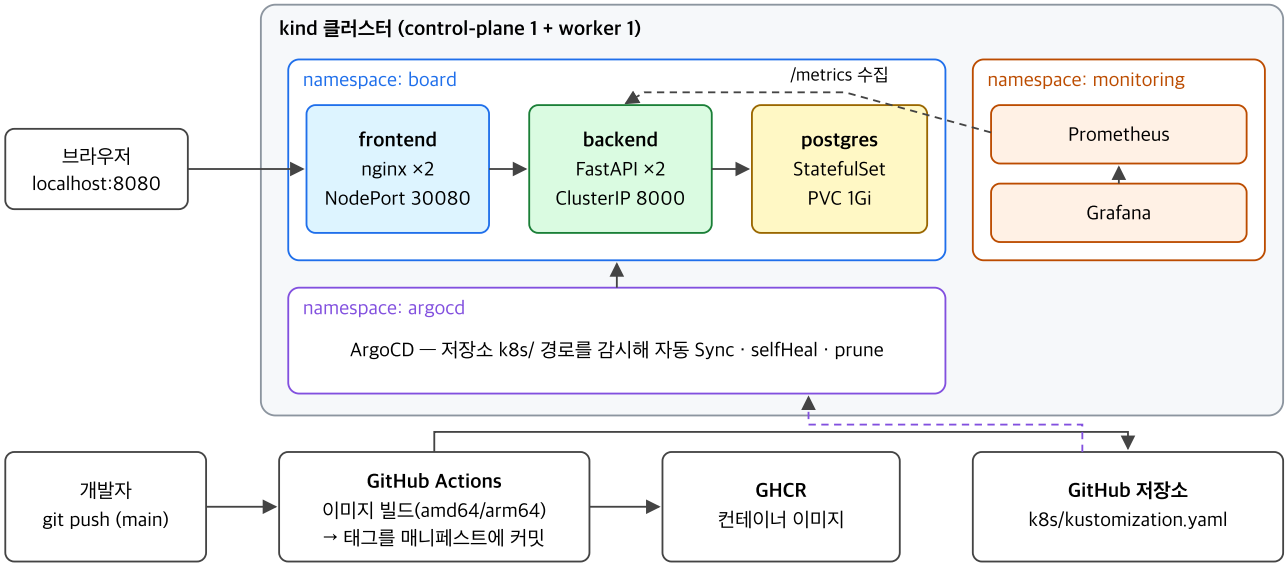
최종 프로젝트 보고서 · 60201707 정은수 · 저장소: https://github.com/jungeuns00/cloud-board

1. 개요

프론트엔드와 백엔드를 독립된 컨테이너와 Kubernetes Deployment로 분리한 게시판 서비스를 로컬 Kubernetes(kind) 클러스터에 배포하고, Git 기반 CI/CD와 모니터링을 구성했다.

평가 항목	구현	내용
게시판 (70)	구현	게시글 목록 조회 · 상세 · 작성 · 수정 · 삭제
CI/CD (30)	구현	main push 시 GitHub Actions가 이미지 빌드·푸시, ArgoCD가 자동 배포
모니터링 (30)	구현	Prometheus가 백엔드 메트릭 수집, Grafana 대시보드 제공
서버리스 (30)	코드만 작성	욕설 필터 Lambda 함수와 백엔드 연동 코드 작성. AWS 배포는 하지 않음

2. 아키텍처



구성 요소

구성 요소	기술	역할
frontend	정적 HTML/JS, nginx	화면 제공. `/api/` 요청은 backend 서비스로 리버스 프록시하므로 브라우저는 프론트엔드 주소만 사용한다.
backend	Python FastAPI, psycopg	게시글 CRUD REST API, `/healthz`, `/metrics` 제공
postgres	PostgreSQL 16	게시글 저장. StatefulSet + PVC로 Pod 재시작 후에도 데이터 유지
CI	GitHub Actions, GHCR	이미지 빌드·푸시, 매니페스트 이미지 태그 갱신
CD	ArgoCD	Git 저장소 상태를 클러스터에 자동 반영 (GitOps)
모니터링	kube-prometheus-stack	Prometheus 수집, Grafana 시각화

API

메서드	경로	설명
GET	/api/posts	게시글 목록 (최신순)
GET	/api/posts/{id}	게시글 상세. 없으면 404
POST	/api/posts	작성 (title, content, author). 성공 시 201
PUT	/api/posts/{id}	수정 (title, content)
DELETE	/api/posts/{id}	삭제. 성공 시 204

3. Dockerfile

backend/Dockerfile

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
# 의존성 레이어를 먼저 만들어 코드만 바뀔 때 캐시를 재사용한다
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app ./app
# root가 아닌 사용자로 실행
RUN useradd --uid 10001 --no-create-home appuser
USER 10001
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

frontend/Dockerfile

```
# 비root(uid 101)로 8080 포트에서 동작하는 nginx 이미지
FROM nginxinc/nginx-unprivileged:1.27-alpine
ENV BACKEND_URL=http://backend:8000
# /etc/nginx/templates/*.template 은 시작 시 envsubst 를 거쳐 conf.d 로 생성된다
COPY default.conf.template /etc/nginx/templates/default.conf.template
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 8080
```

두 이미지 모두 비root 사용자로 실행한다. 프론트엔드는 백엔드 주소를 환경 변수 `BACKEND_URL` 로 받아 이미지를 다시 빌드하지 않고도 환경별로 바꿀 수 있다.

4. Kubernetes 구성

리소스	설정
Deployment frontend	replicas 2, readinessProbe `/healthz`, Service NodePort 30080 (kind 포트 매핑으로 호스트 8080에 노출)
Deployment backend	replicas 2, readiness/liveness `/healthz` (DB 연결 확인), ConfigMap과 Secret으로 DB 접속 정보 주입, Service ClusterIP
StatefulSet postgres	replicas 1, volumeClaimTemplates 1Gi, headless Service, `pg_isready` readinessProbe
Kustomize	`k8s/kustomization.yaml`의 `images` 항목으로 이미지 태그를 관리

5. CI/CD

1. main 브랜치에 `backend/` 또는 `frontend/` 변경이 push되면 `.github/workflows/ci.yaml`이 실행된다.
2. **build** 잡이 두 서비스를 병렬로 빌드해 `ghcr.io/jungeuns00/cloud-board-{backend,frontend}:<커밋 SHA>`로 푸시한다. 로컬 클러스터가 Apple Silicon이므로 amd64/arm64 멀티 아키텍처로 빌드한다.
3. **deploy** 잡이 `kustomize edit set image`로 매니페스트의 태그를 새 SHA로 바꿔 커밋한다. 워크플로는 클러스터에 직접 접근하지 않는다.
4. ArgoCD Application이 저장소의 `k8s/` 경로를 감시하다가 변경을 감지해 자동으로 Sync하고, Deployment가 롤링 업데이트된다.

ArgoCD에는 `automated`(prune, selfHeal)를 설정해, 클러스터에서 리소스를 수동으로 바꾸거나 지워도 Git 상태로 되돌아간다.

[스크린샷] GitHub Actions 실행 성공 화면 / ArgoCD UI의 cloud-board 애플리케이션 (Synced · Healthy)

6. 모니터링

- Helm으로 kube-prometheus-stack을 `monitoring` 네임스페이스에 설치했다.

- 백엔드는 `prometheus-fastapi-instrumentator`로 요청 수·응답 시간을 `/metrics`에 노출하고, 게시글 작성/수정/삭제 횟수를 커스텀 카운터(`board_posts_created_total` 등)로 추가했다.
- `ServiceMonitor`로 backend 서비스를 수집 대상에 등록했다 (15초 주기).
- Grafana 대시보드 "Cloud Board"를 ConfigMap으로 등록했다: 실행 중인 Pod 수, 누적 게시글 수, 5xx 오류율, 요청 처리량, 응답 시간 p95, 작성/수정/삭제 추이, Pod 메모리 사용량.

[스크린샷] Grafana "Cloud Board" 대시보드 / Prometheus Targets 화면

7. 서버리스 함수 (코드만 작성)

`serverless/lambda_function.py`는 제목과 내용에서 금칙어를 `*`로 마스킹해 돌려주는 Lambda 함수다. 백엔드는 환경 변수 `FILTER_URL`이 설정되어 있으면 게시글 작성·수정 시 이 함수를 호출하고, 호출이 실패하면 원문을 그대로 저장한다. 함수 로직은 로컬에서 직접 호출해 확인했으나 AWS에 배포하지는 않았으므로 현재 클러스터에서는 필터링이 꺼져 있다.

8. 동작 확인

확인 항목	결과
게시글 작성 (POST)	201, 생성된 글 반환
게시글 수정 (PUT)	200, 제목·내용·수정 시각 변경
게시글 삭제 (DELETE)	204, 이후 조회 시 404
목록 조회 (GET)	남은 글만 최신순으로 반환
빈 제목으로 작성	422 (입력 검증)
main push → 이미지 빌드 → 태그 커밋	워크플로 성공, `deploy: <SHA>` 커밋 자동 생성
ArgoCD 자동 배포	Application 등록 후 board 네임스페이스 리소스 자동 생성

[스크린샷] 게시판 화면 / `kubectl -n board get deploy,pod,svc,pvc` / psql 조회 결과

9. 제한사항

- **단일 노드 DB**: PostgreSQL이 복제 없이 Pod 1개로 동작한다. Pod나 노드 장애 시 서비스가 중단되며 백업도 구성하지 않았다.
- **비밀 정보 관리**: DB 비밀번호가 데모용 값으로 Git 저장소의 Secret 매니페스트에 평문으로 들어 있다. 실제 운영에서는 Sealed Secrets나 External Secrets가 필요하다.
- **인증 없음**: 로그인 기능이 없어 누구나 모든 글을 수정·삭제할 수 있다.

- **로컬 클러스터**: kind는 Docker 컨테이너를 노드로 쓰므로 LoadBalancer, Ingress, 외부 도메인이 없다. 프론트엔드는 NodePort, ArgoCD와 Grafana는 port-forward로 접근한다.
- **배포 지연**: ArgoCD가 저장소를 주기적으로(기본 3분) 확인하므로 push 후 반영까지 수 분이 걸릴 수 있다. Webhook은 로컬 클러스터라 구성하지 않았다.
- **모니터링 범위**: Alertmanager 알림은 꺼 두었고, kind 환경에서 수집할 수 없는 etcd·scheduler 등 컨트롤 플레인 메트릭은 제외했다. 로그 수집은 구성하지 않았다.
- **자동 확장 없음**: HPA를 구성하지 않아 replica 수가 고정이다.
- **서버리스**: Lambda 함수는 코드만 작성했고 배포·연동 시연은 하지 않았다.